

Clase 6

Exploración y consumo de APIs

IIC2513 - Tecnologías y Aplicaciones Web

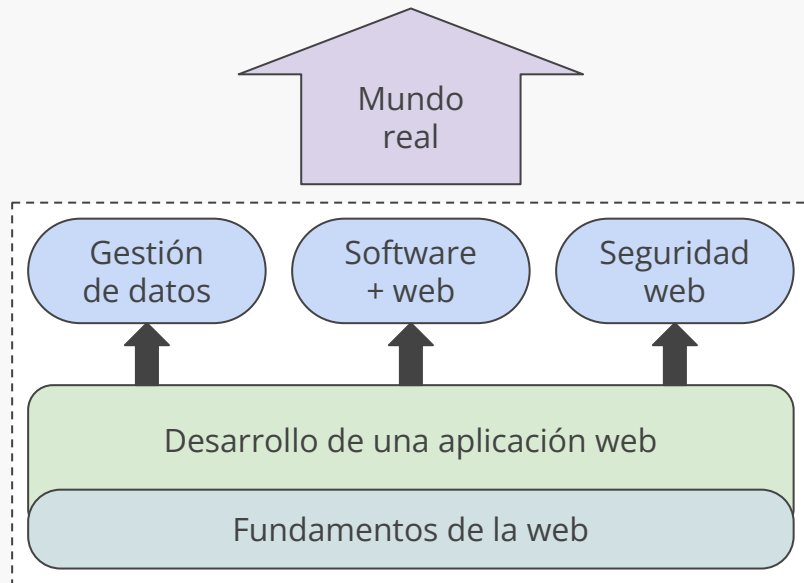
Antonio Ossa Guerra
aaossa@ing.puc.cl

1. Mañana sí hay ayudantía
2. ¿Veamos la ECA?

Anuncios

Contenidos del curso

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real

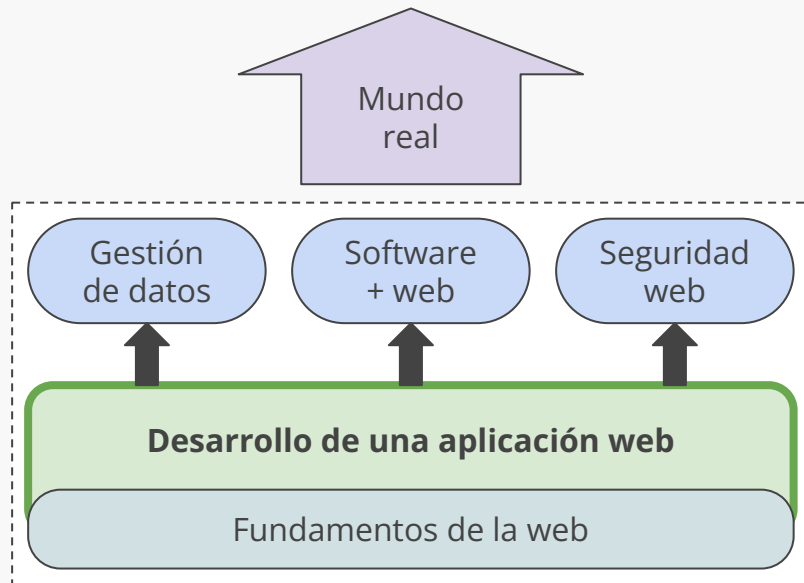


Contenidos del curso

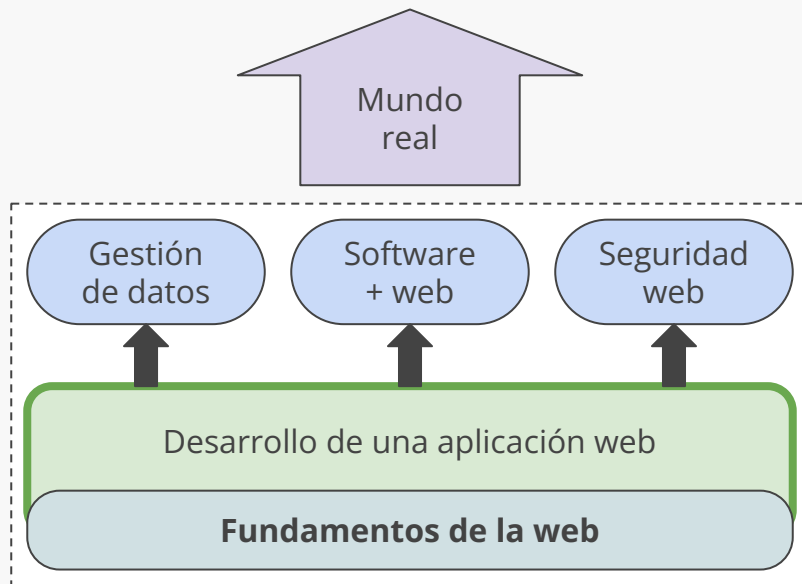
Desarrollo de una aplicación web

Veremos herramientas como **React**, **Node.js**, Koa y Express, y técnicas como CSS avanzado y el uso de *Design Systems*

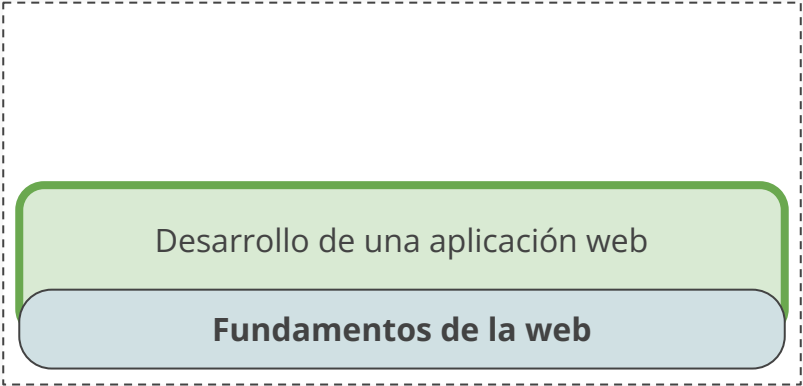
... para desarrollar **aplicaciones modernas** tanto en el lado del **cliente** (front-end) como en el lado del servidor (back-end)



Contenidos del curso



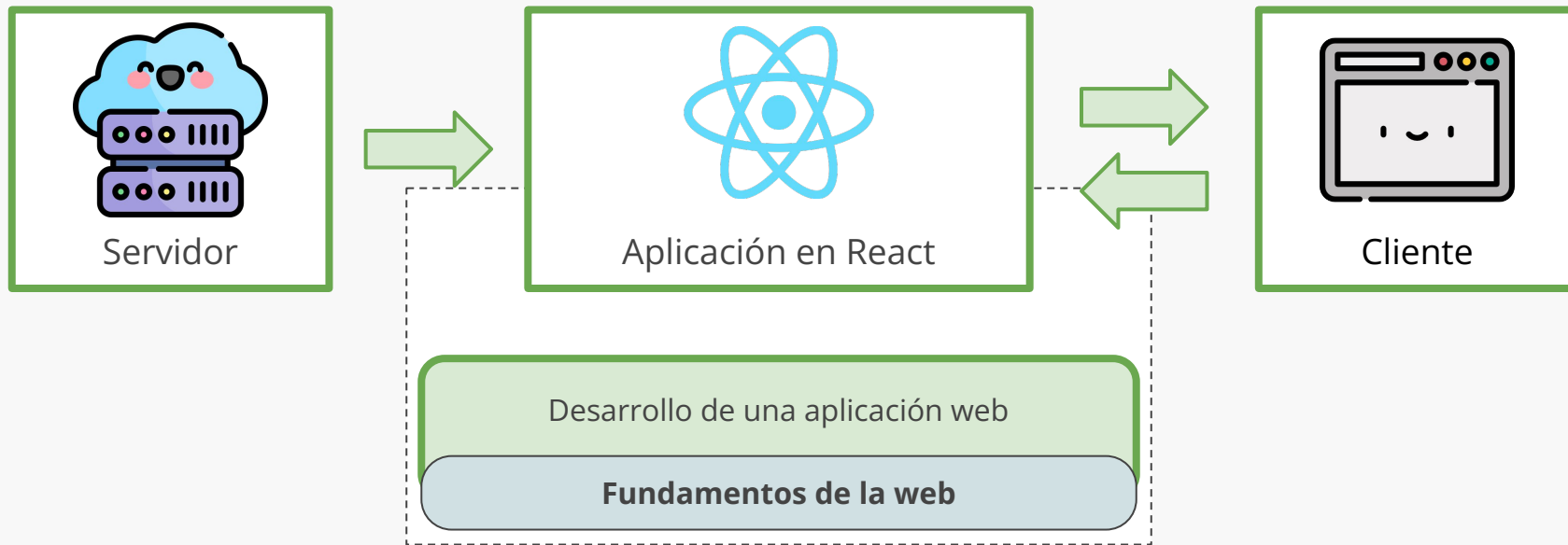
Contenidos del curso



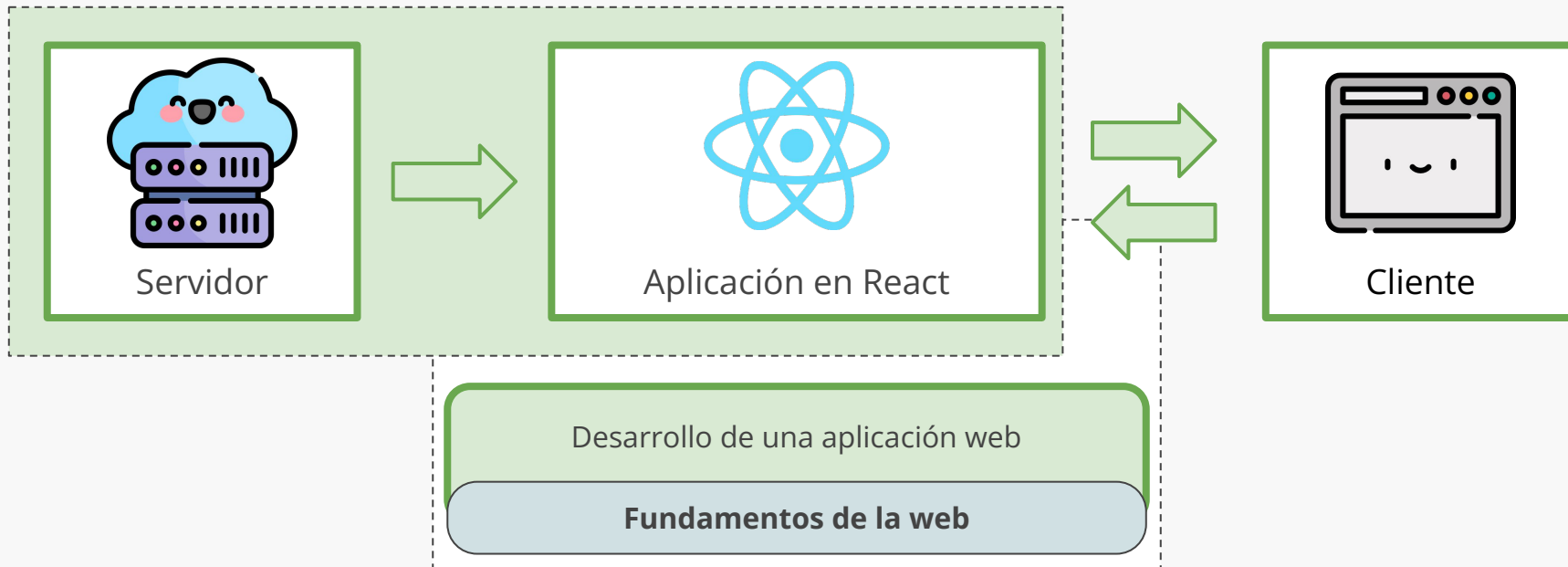
Desarrollo de una aplicación web

Fundamentos de la web

Contenidos del curso



Contenidos del curso

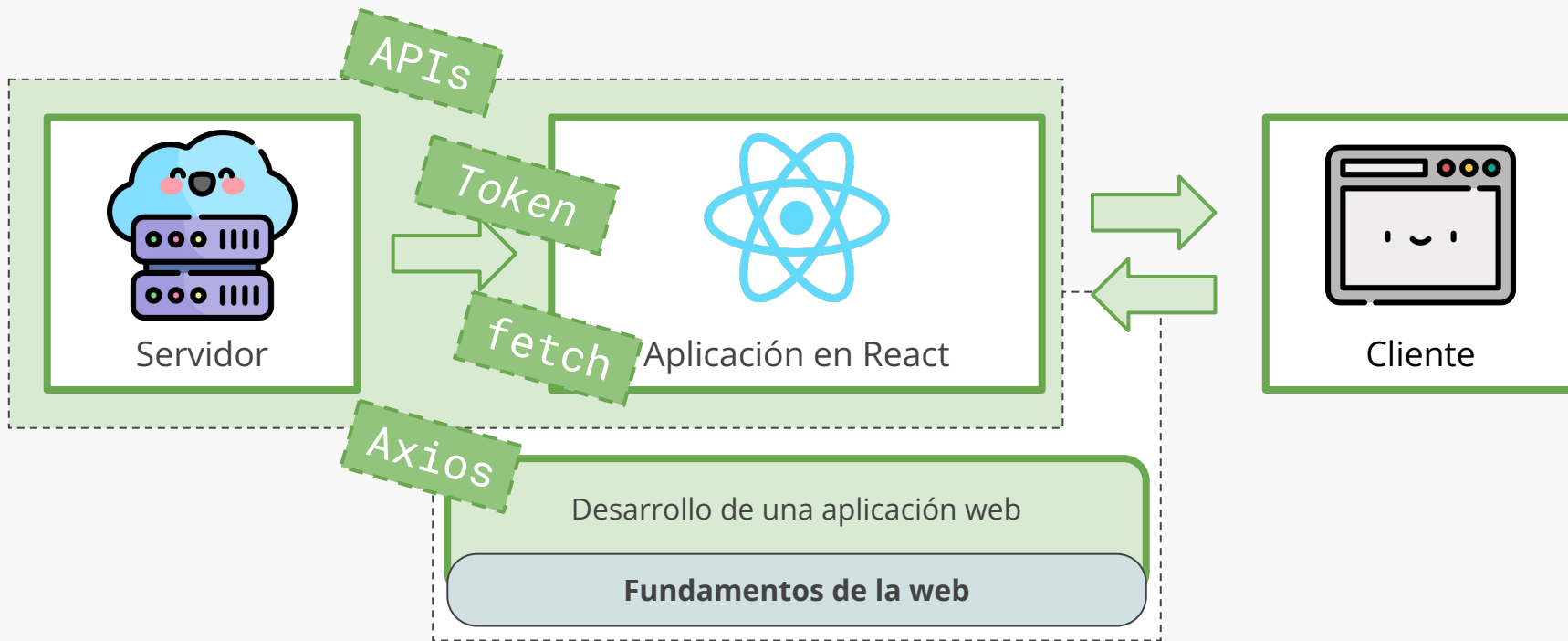


Contenidos del curso

Pregunta# ¿Cómo podemos acceder a recursos desde nuestra aplicación en React?

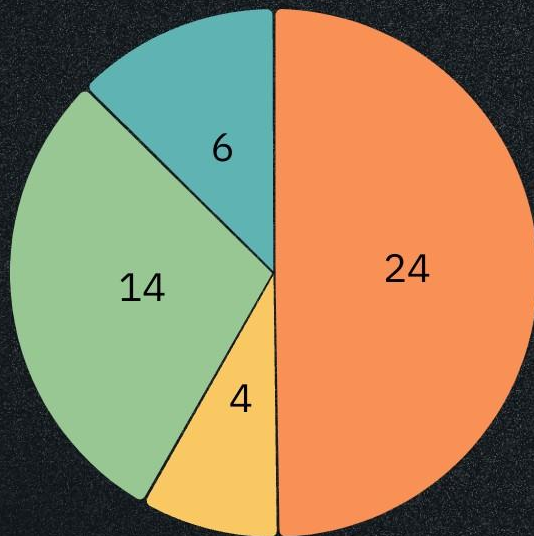


Contenidos del curso



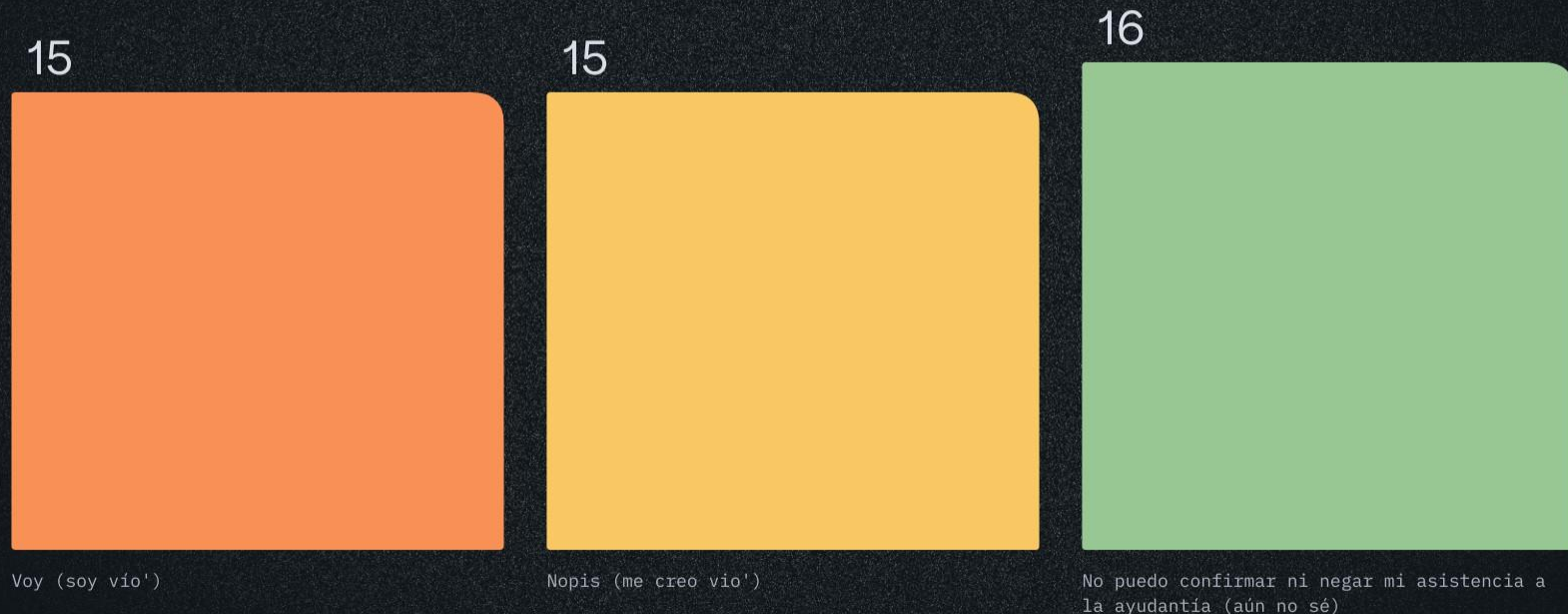
Menti*#Vamos con el Menti del día*

¿Cómo te fue con el Control 1? 🙄



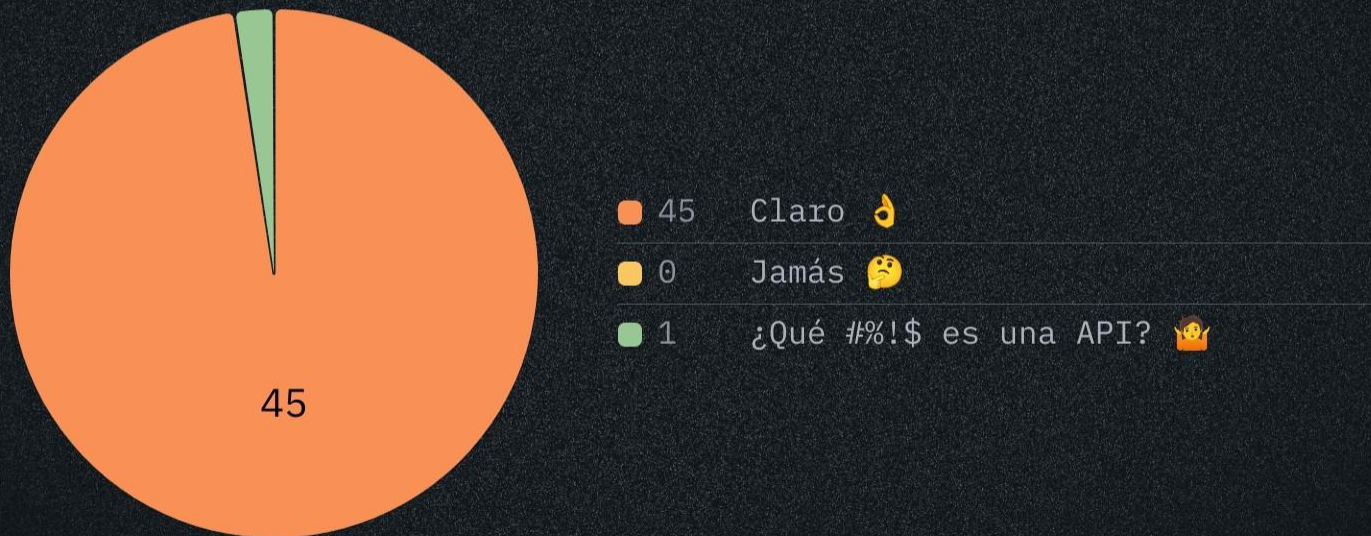
- 24 Aún no lo hago 🕒
- 4 Lo intenté una vez 👍
- 14 Completado ✅
- 6 La encontré fácil 😎

¿Vas a ir a la ayudantía de React? 🙄



Hablemos de APIs

¿Has ocupado alguna API?



¡Gracias por responder!

Mañana hay ayudantía de "React".
Los ayudantes del curso tienen
una ayudantía muy útil preparada

Agenda

✨ APIs

Explorando APIs

Consumiendo APIs

¿Qué es una API?

API = Application Programming Interface

Es una **interfaz** o un intermediario entre una aplicación o software y quien desee interactuar con la aplicación por medio de código:

- Se dice que una aplicación **“expone”** funcionalidades mediante una API
- Quien desee acceder a estas funcionalidades debe **“consumir”** la API

¿Qué es una API?

Establece **comunicación entre aplicaciones**, ya que puede ser otra aplicación la que consume la API (por eso es *Application **Programming** Interface*):

```
expect(received).toBe(expected);
```

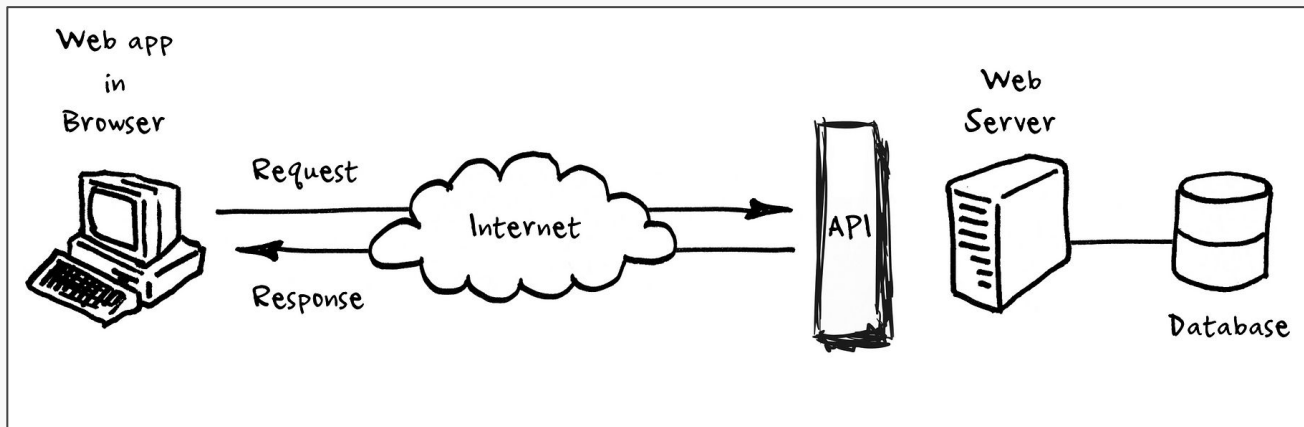
```
Math.max(5, 2, 100, 33);
```

```
GET https://api.github.com/orgs/IIC2513-2021-1/repos
```

¿Qué es una API Web?

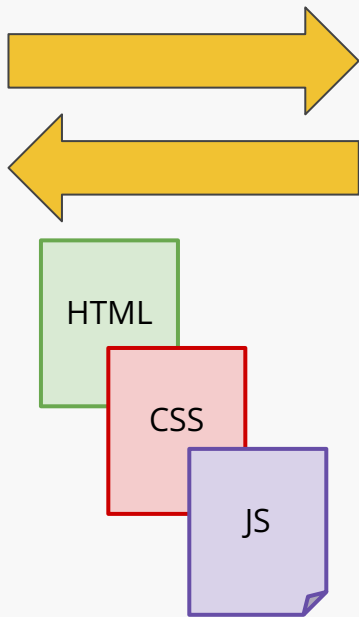
API a la que se accede **usando la infraestructura Web** (protocolo HTTP):

- Genéricamente, cliente hace un *request* y recibe datos como respuesta
- No necesariamente un documento HTML (muchas veces veremos JSON)

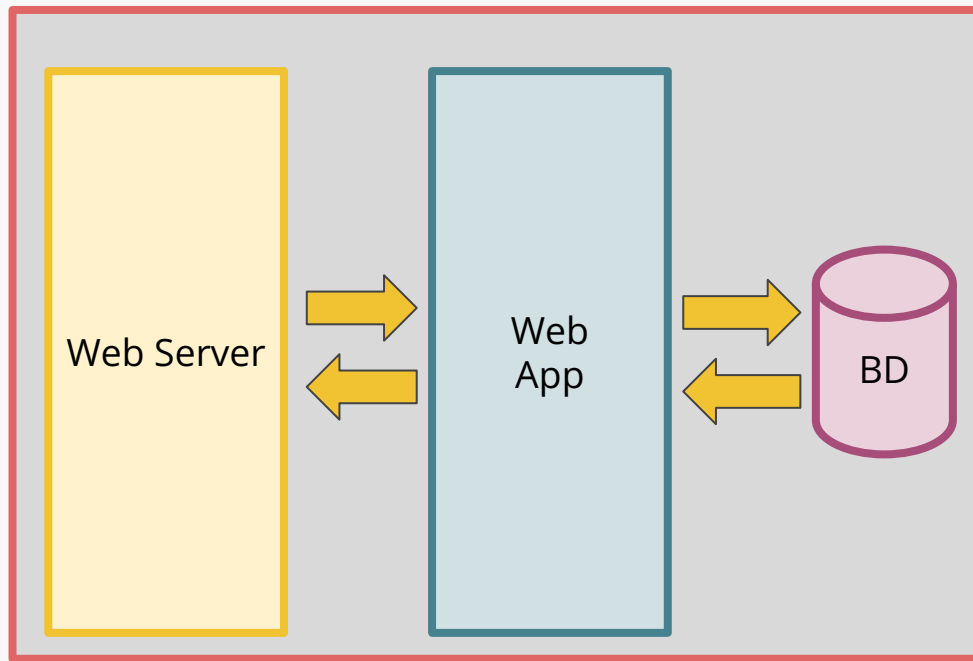


Aplicación web moderna

Cliente



Servidor

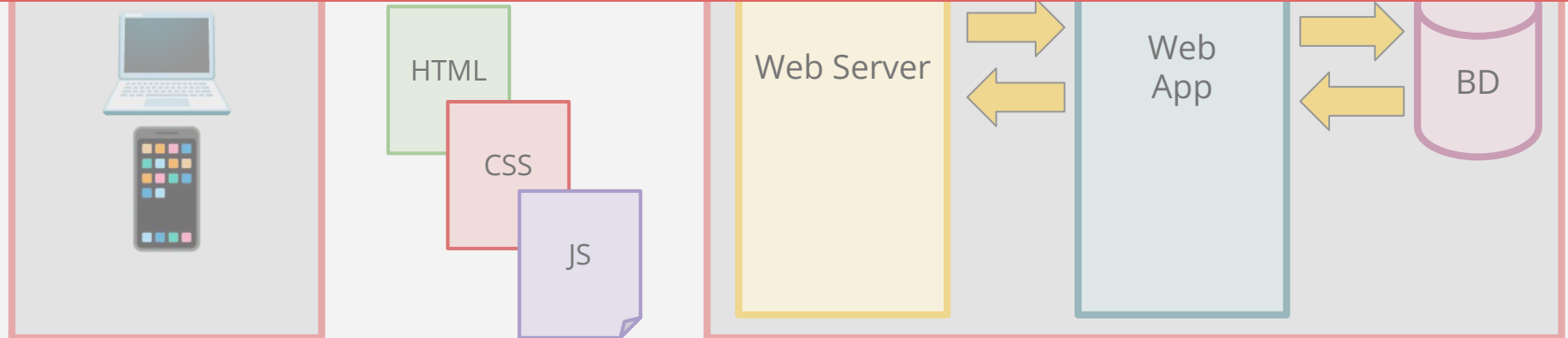


Aplicación web moderna

Cliente

Servidor

Clase 8#Desarrollando servidores

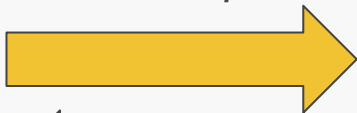


¿Dónde está la API Web?

Cliente



HTTP *request*



HTTP *response*

Servidor

API

Las URLs suelen llamarse ***endpoints***

APIs Web

Se conocen de forma genérica como “web services”:

- Las respuestas contienen **datos estructurados** que son presentados en una manera estándar, de forma que podamos asumir su estructura
- Diferentes tipos de aplicaciones pueden interpretar sus respuestas
- Los formatos más utilizados en una API son JSON y XML

XML	vs.	JSON
<pre>1 <?xml version="1.0" encoding="UTF-8"?> 2 <endereco> 3 <cep>31270901</cep> 4 <city>Belo Horizonte</city> 5 <neighborhood>Pampulha</neighborhood> 6 <service>correios</service> 7 <state>MG</state> 8 <street>Av. Presidente Antônio Carlos, 6627</street> 9 </endereco></pre>		<pre>1 { 2 "endereco": { 3 "cep": "31270901", 4 "city": "Belo Horizonte", 5 "neighborhood": "Pampulha", 6 "service": "correios", 7 "state": "MG", 8 "street": "Av. Presidente Antônio Carlos, 6627" 9 } 10 }</pre>

Agenda

APIs



CRUD

REST

Autenticación

Explorando APIs

Consumiendo APIs

¿Qué entendemos por CRUD?

Son las **operaciones** que podemos realizar sobre un recurso

Create

Read

Update

Delete

CRUD en contexto web

R	GET	/users	Obtener colección de recursos
C	POST	/users	Agregar un recurso a colección
R	GET	/users/2	Obtener un recurso
U	PUT	/users/2	Definir atributos para un recurso
U	PATCH	/users/2	Modificar atributos de un recurso
D	DELETE	/users/2	Eliminar un recurso

En general, todas las aplicaciones web y APIs **incluyen operaciones CRUD**

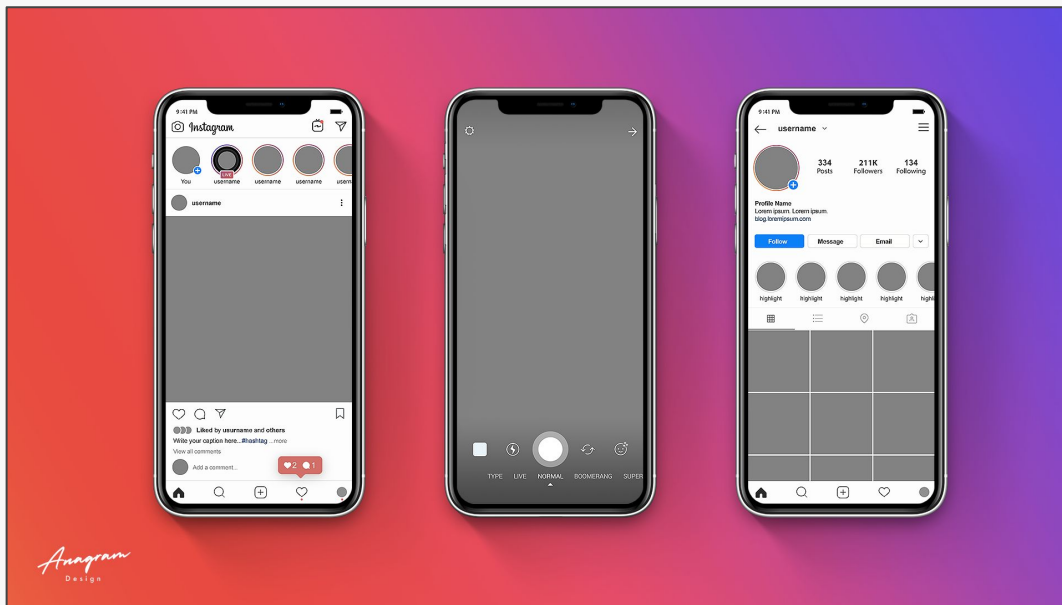
Ejemplo: Instagram

C: Subir posts o stories

R: Cargar timeline

U: Editar post propio

D: Eliminar post propio



Ejemplo: Instagram

R	GET	/stories	Obtener colección de historias
C	POST	/stories	Agregar una nueva historia
R	GET	/posts/1	Obtener un post en específico
U	PUT	/posts/2	Definir atributos para un post
U	PATCH	/posts/2	Modificar atributos de un post
D	DELETE	/posts/2	Eliminar un post

Agenda

APIs

CRUD



REST

Autenticación

Explorando APIs

Consumiendo APIs

¿Qué es REST?

REST = REpresentational State Transfer

Estilo arquitectónico que define una **serie de principios** (o restricciones) que debe cumplir un **servicio web** para que sea escalable y fácil de usar*

([Disertación publicada](#) por Roy Fielding en el año 2000 para conseguir su grado de PhD)

- En su definición, utiliza los conceptos de **recursos y representaciones** de recursos
- Se apoya fuertemente en los **métodos HTTP** para **especificar acciones**

Principios REST

Un servicio que cumpla con todo esto puede ser considerado **RESTful**

- Client-server
- Stateless
- Cacheable
- Uniform interface
 - Resource-based
 - Manipulation of resources through representations
 - Self-descriptive messages
 - Hypermedia as the engine of application state (HATEOAS)
- Layered system
- Code on demand (optional)

Principios REST

- **Client-server** deben estar separados y poder evolucionar independientemente
- **Stateless** cada request del cliente debe contener toda la información necesaria para completarla
- **Cacheable** una respuesta debería poder indicar si es cacheable o no
- **Uniform interface**
 - **Resource-based**
cada recurso se identifica únicamente
 - **Manipulation of resources through representations**
las representaciones son uniformes en la respuesta
 - **Self-descriptive messages**
cada recurso debe llevar suficiente información para poder procesar el mensaje
 - **Hypermedia as the engine of application state (HATEOAS)**
por medio de hipervínculos, el cliente debería poder interactuar con todos los recursos
- **Layered system** un cliente no debería poder distinguir si hay una implementación por capas, es transparente
- **Code on demand (optional)** envío de script del servidor al cliente para indicar cómo ejecutar alguna acción

¿Cómo se ve un RESTful web service?

GET /repos/{owner}/{repo}/issues/{issue_number}

REST se apoya fuertemente en HTTP

- Scope en **path** (endpoints)
- Acción u operación en **método HTTP**
- Datos en **body** (payload)

POST /repos/{owner}/{repo}/issues

cURL

```
curl -L \
-X POST \
-H "Accept: application/vnd.github.v3+json" \
-H "Authorization: token <token>" \
-H "X-GitHub-Api-Version: 2022-11-28" \
https://api.github.com/repos/{owner}/{repo}/issues \
-d '{"title": "this.", "body": "a"}
```

```
// URL
const urlParams = {
  owner: 'OWNER',
  repo: 'REPO',
  issue_number: 'ISSUE_NUMBER',
};

// Headers
const headers = {
  'X-GitHub-Api-Version': '2022-11-28',
};
```

¿Cómo se ve un RESTful web service?

R	GET	/users	Obtener colección de recursos
C	POST	/users	Agregar un recurso a colección
R	GET	/users/2	Obtener un recurso
U	PUT	/users/2	Definir atributos para un recurso
U	PATCH	/users/2	Modificar atributos de un recurso
D	DELETE	/users/2	Eliminar un recurso

- Cada *endpoint* es independiente del resto
- No existe la redirección
- Un caso de uso puede significar consumir varios *endpoints*

¿Cómo se ve un RESTful web service?

R	GET	/users	Obtener colección de recursos
C	POST	/users	Agregar un recurso a colección
D	GET	/users/2	Obtener un recurso
D	DELETE	/users/2	Eliminar un recurso

Pregunta#¿Diferencia entre CRUD y REST?

- Cada *endpoint* es independiente del resto
- No existe la redirección
- Un caso de uso puede significar consumir varios *endpoints*

GitHub API REST docs: Documentación de la API de GitHub



GitHub Docs

Version: Free, Pro, & Team ▾

Search GitHub Docs



← Home

REST API

API Version: 2022-11-28 (latest) ▾

Quickstart

About the REST API ▾

Using the REST API ▾

Authentication ▾

Guides ▾

Actions ▾

Activity ▾

Apps ▾

Billing ▾

Branches ▾

Checks ▾

Classroom ▾

Code scanning ▾

Security settings ▾

Codes of conduct ▾

Codespaces ▾

The REST API is now versioned. For more information, see "[About API versioning](#)."

GitHub REST API documentation

Create integrations, retrieve data, and automate your workflows with the GitHub REST API.

Overview

Quickstart

Start here [View all →](#)

About the REST API

Get oriented to the REST API documentation.

Getting started with the REST API

Learn how to use the GitHub REST API.

Authenticating to the REST API

You can authenticate to the REST API to access more endpoints and have a higher rate limit.

Popular

Rate limits for the REST API

Learn about REST API rate limits, how to avoid exceeding them, and what to do if you do exceed them.

Troubleshooting the REST API

Learn how to diagnose and resolve common problems for the REST API.

Scripting with the REST API and JavaScript

What's new [View all →](#)

GitHub Issues & Projects: REST API support for issue types

March 18

Fine-grained PATs are now generally available

March 18

GitHub Issues & Projects: API support for issues advanced search and more!

March 06

Agenda

APIs

CRUD

REST



Autenticación

Explorando APIs

Consumiendo APIs

Principios REST

- **Client-server** deben estar separados y poder evolucionar independientemente
- **Stateless** cada request del cliente debe contener toda la información necesaria para completarla
- **Cacheable** una respuesta debería poder indicar si es cacheable o no
- **Uniform interface**
 - **Resource-based**
cada recurso se identifica únicamente
 - **Manipulation of resources through representations**
las representaciones son uniformes en la respuesta
 - **Self-descriptive messages**
cada recurso debe llevar suficiente información para poder procesar el mensaje
 - **Hypermedia as the engine of application state (HATEOAS)**
por medio de hipervínculos, el cliente debería poder interactuar con todos los recursos
- **Layered system** un cliente no debería poder distinguir si hay una implementación por capas, es transparente
- **Code on demand (optional)** envío de script del servidor al cliente para indicar cómo ejecutar alguna acción

HTTP stateless

Si una *request* es **independiente de la siguiente...**

¿Cómo podemos **iniciar sesión**
y ser **identificados** en cada *request* a la API?

HTTP stateless

Si una *request* es **independiente de la siguiente...**

¿Cómo podemos **iniciar sesión**
y ser **identificados** en cada *request* a la API?

Volveremos a visitar esto
cuando hablemos de *cookies* y *sessions*

HTTP stateless

Clase 14#*Cookies y sesiones*

¿Cómo podemos **iniciar sesión**
y ser **identificados** en cada *request* a la API?

Volveremos a visitar esto
cuando hablemos de *cookies y sessions*

Autenticación API RESTful

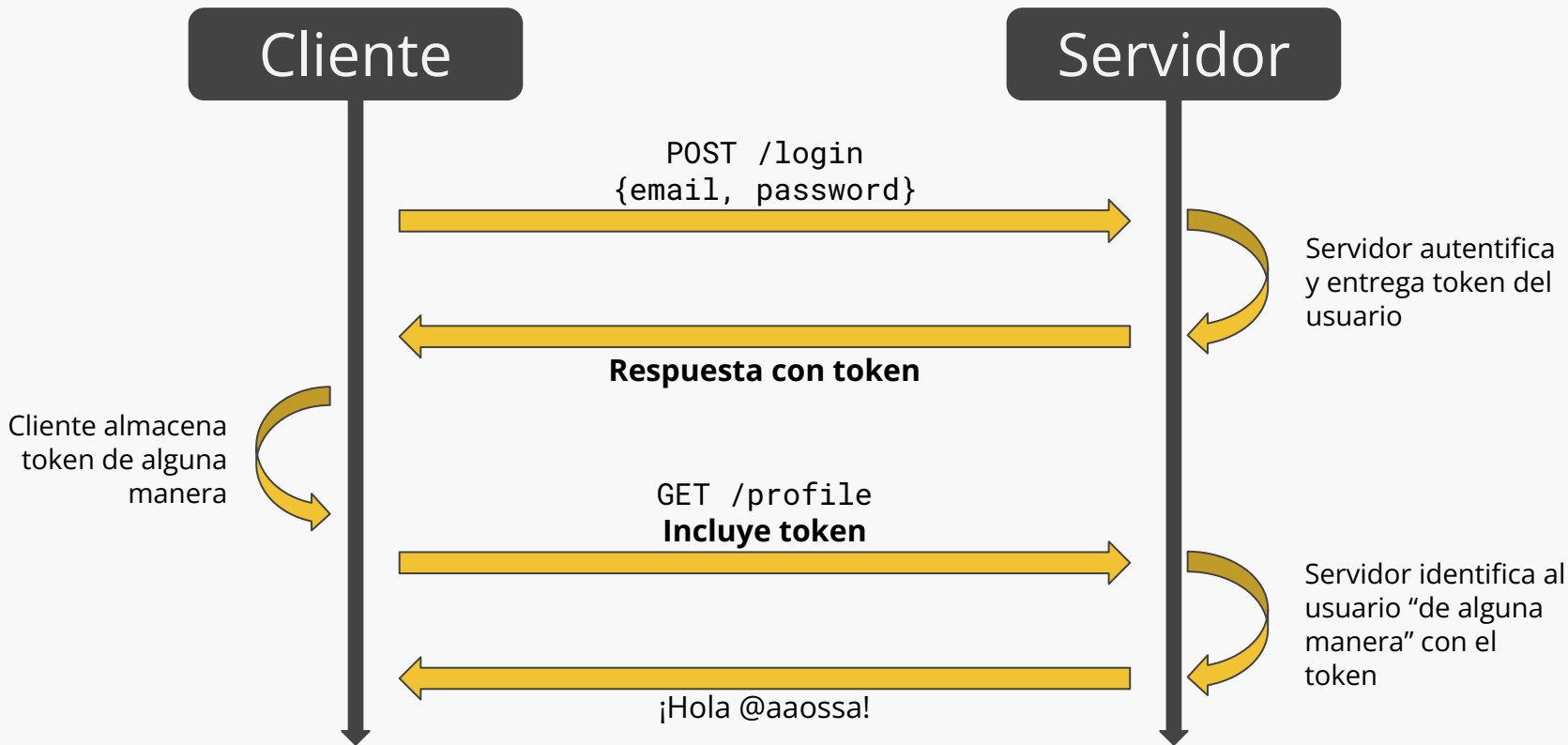
Access token

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

String con **caracteres aleatorios*** que actúa como credencial de seguridad para identificar a un usuario, su grupo, sus permisos y/o alguna aplicación, por lo que sirve para **autenticar a un usuario**

*: a veces son con apariencia aleatoria (lo veremos más adelante en, por ejemplo, JWTs)

Flujo autenticación



HTTP request



HTTP request



HTTP stateless

Si una *request* es **independiente de la siguiente...**

¿Cómo podemos “**cerrar sesión**”?

¿Y el “cierre de sesión”?

Hay que “**revocar**” el **token**, para lo que existen distintas estrategias:

- Mantener una lista de *tokens* válidos / no válidos
- Recordar cuándo expira un *token* en el servidor
- Codificar la validez del *token* en el mismo *token*
- ... y otros mecanismos

Agenda

APIs



Explorando APIs

Consumiendo APIs

Documentación API

La mayoría de las APIs REST incluyen una documentación que puede entenderse como un manual de uso de la API que suele incluir ejemplos

La calidad de la documentación influye en la experiencia de los consumidores de la API

 GitHub Docs

Version: Free, Pro, & Team ▾

Search GitHub Docs

Q

🌐

☰ REST API

The REST API is now versioned. For more information, see "About API versioning."

GitHub REST API documentation

Create integrations, retrieve data, and automate your workflows with the GitHub REST API.

Overview

Quickstart

Start here ▾ [View all →](#)

About the REST API

Get oriented to the REST API documentation.

Getting started with the REST API

Learn how to use the GitHub REST API.

Authenticating to the REST API

You can authenticate to the REST API to access more endpoints and have a higher rate limit.

Best practices for using the REST API

Follow these best practices when using GitHub's API.

Popular

Rate limits for the REST API

Learn about REST API rate limits, how to avoid exceeding them, and what to do if you do exceed them.

Troubleshooting the REST API

Learn how to diagnose and resolve common problems for the REST API.

Scripting with the REST API and JavaScript

Write a script using the Octokit.js SDK to interact with the REST API.

Keeping your API credentials secure

Follow these best practices to keep your API credentials and tokens secure.

What's new ▾ [View all →](#)

Deprecation of cvss field in security advisories API

March 27

GitHub Issues & Projects: REST API support for issue types

March 18

Fine-grained PATs are now generally available

March 18

Ejemplos de docs

Veamos un ejemplo de buena documentación a través de la documentación de [GitHub API \(commits\)](#) en particular con listar commits ([docs](#))

Otros buenos ejemplos:

- [Stripe API](#)
- [Canvas LMS API](#)

 GitHub Docs

Version: Free, Pro, & Team

Search GitHub Docs

REST API / Commits / Commits

List commits

Signature verification object

The response will include a `verification` object that describes the result of verifying the commit's signature. The following fields are included in the `verification` object:

Name	Type	Description
<code>verified</code>	<code>boolean</code>	Indicates whether GitHub considers the signature in this commit to be verified.
<code>reason</code>	<code>string</code>	The reason for verified value. Possible values and their meanings are enumerated in table below.
<code>signature</code>	<code>string</code>	The signature that was extracted from the commit.
<code>payload</code>	<code>string</code>	The value that was signed.
<code>verified_at</code>	<code>string</code>	The date the signature was verified by GitHub.

These are the possible values for `reason` in the `verification` object:

Value	Description
<code>expired_key</code>	The key that made the signature is expired.
<code>not_signing_key</code>	The "signing" flag is not among

Code samples for "List commits"

Request example

```
GET /repos/{owner}/{repo}/commits
```

cURL JavaScript GitHub CLI

```
curl -L \
-H "Accept: application/vnd.github+json" \
-H "Authorization: Bearer <YOUR-TOKEN>" \
-H "X-GitHub-Api-Version: 2022-11-28" \
https://api.github.com/repos/OWNER/REPO/commits
```

Response

Example response Response schema

Status: 200

```
{
  "url": "https://api.github.com/repos/octocat/Hello-World/commits/6dc09b5b57875f334f61aebd695e2e4193db5e",
  "sha": "6dc09b5b57875f334f61aebd695e2e4193db5e",
  "node_id": "MDY6Q29tbWl0eNMrjYjASyJvYnR0c09b5b57875f334f61aebd695e2e4193db5e",
  "html_url": "https://github.com/octocat/Hello-World/commit/6dc09b5b57875f334f61aebd695e2e4193db5e",
  "comments_url": "https://api.github.com/repos/octocat/Hello-World/commits/6dc09b5b57875f334f61aebd695e2e4193db5e/comments",
  "commit": {
    "url": "https://api.github.com/repos/octocat/Hello-World/git/commits/6dc09b5b57875f334f61aebd695e2e4193db5e",
    "author": {
      "name": "Monalisa Octocat",
      "email": "support@github.com",
      "date": "2011-04-14T16:00:49Z"
    },
    "message": "Initial commit"
  }
}
```

Ejemplos de docs

¿Qué suelen incluir?

- Método HTTP + *endpoint (path)*
- Parámetros de *request*
- *Response*
- *Errors*
- Ejemplos



GitHub Docs

Version: Free, Pro, & Team

Search GitHub Docs



REST API / Commits / Commits

List commits

Signature verification object

The response will include a `verification` object that describes the result of verifying the commit's signature. The following fields are included in the `verification` object:

Name	Type	Description
<code>verified</code>	<code>boolean</code>	Indicates whether GitHub considers the signature in this commit to be verified.
<code>reason</code>	<code>string</code>	The reason for verified value. Possible values and their meanings are enumerated in table below.
<code>signature</code>	<code>string</code>	The signature that was extracted from the commit.
<code>payload</code>	<code>string</code>	The value that was signed.
<code>verified_at</code>	<code>string</code>	The date the signature was verified by GitHub.

These are the possible values for `reason` in the `verification` object:

Value	Description
<code>expired_key</code>	The key that made the signature is expired.
<code>not_signing_key</code>	The "signing" flag is not among

Code samples for "List commits"

Request example

GET /repos/{owner}/{repo}/commits

cURL JavaScript GitHub CLI

```
curl -L \
-H "Accept: application/vnd.github+json" \
-H "Authorization: Bearer <YOUR-TOKEN>" \
-H "X-GitHub-Api-Version: 2022-11-28" \
https://api.github.com/repos/OWNER/REPO/commits
```

Response

Example response Response schema

Status: 200

```
{
  "url": "https://api.github.com/repos/octocat/Hello-World/commits/6dc09b5b57875f334f61aebd695e2e4193db5e",
  "sha": "6dc09b5b57875f334f61aebd695e2e4193db5e",
  "node_id": "MDY6Q29tbWl0eNMrjYjASyYjVlNTc4NzVhZmMzM0ZjYXVhbnVlZWQ2OTVlMmU0MTkzZGI1ZQ==",
  "html_url": "https://github.com/octocat/Hello-World/commit/6dc09b5b57875f334f61aebd695e2e4193db5e",
  "comments_url": "https://api.github.com/repos/octocat/Hello-World/commits/6dc09b5b57875f334f61aebd695e2e4193db5e/comments",
  "commit": {
    "url": "https://api.github.com/repos/octocat/Hello-World/git/commits/6dc09b5b57875f334f61aebd695e2e4193db5e",
    "author": {
      "name": "Monalisa Octocat",
      "email": "support@github.com",
      "date": "2011-04-14T16:00:49Z"
    },
    "message": "My commit message"
  }
}
```

Ejemplo de GET *request* a API de GitHub

Veamos un ejemplo accediendo a la lista de *commits* de un repo en GitHub ([docs](#)):

<i>Request line</i>	GET https://api.github.com/repos/IIC2513/Syllabus-S1/commits
<i>Request header</i>	Accept: application/vnd.github+json Authorization: Bearer <YOUR-TOKEN> X-GitHub-API-Version: 2022-11-28
<i>Request body</i> (Opcional)	

Ejemplo de POST *request* a API de GitHub

Veamos un ejemplo para crear una *issue* en un repo en GitHub ([docs](#)):

<i>Request line</i>	POST https://api.github.com/repos/IIC2513/Syllabus-S1/issues
<i>Request header</i>	Accept: application/vnd.github+json Authorization: Bearer <YOUR-TOKEN> X-GitHub-API-Version: 2022-11-28
<i>Request body</i> (Opcional)	<pre>{ "title": "Found a bug" "body": "I\''m having a problem with this.", "assignees": ["octocat"], "milestone": 1, "labels": ["bug"] }</pre>

Formato OpenAPI

[Estándar](#) que automatiza el proceso de documentación de una API REST, utilizando archivos JSON o YAML

- Estructura definida
- Entendida por humanos y máquinas
- Ejemplo: [GitHub API](#)

Existen herramientas para ayudarnos a construir documentación a partir de este formato, por ejemplo, [Swagger](#)

Fuente: [Basic Structure | Swagger Docs](#)

```
openapi: 3.0.4
info:
  title: Sample API
  description: Optional multiline or single-line
description in [CommonMark](http://commonmark.org/help/)
or HTML.
  version: 0.1.9

servers:
  - url: http://api.example.com/v1
    description: Optional server description, e.g. Main
(production) server
  - url: http://staging-api.example.com
    description: Optional server description, e.g.
Internal staging server for testing

paths:
  /users:
    get:
      summary: Returns a list of users.
      description: Optional extended description in
CommonMark or HTML.
      responses:
        "200": # status code
          description: A JSON array of user names
          content:
            application/json:
              schema:
                type: array
                items:
                  type: string
```

Bonus: ¿y APIs no documentadas?

Toda aplicación web moderna funciona con un *front-end* (lo que vemos en nuestro navegador) y obtiene el contenido desde su *back-end* **por medio de requests HTTP**, conectándose a *endpoints* disponibles en algún servidor

Podemos explorar estas conexiones usando *DevTools*, lo que nos permitirá identificar los *endpoints* y los métodos HTTP utilizados para contactar al servidor. Lo que estamos haciendo es **ingeniería inversa** (tendremos toda una clase de esto)

Este artículo de **Julia Evans** llamado **"How to use undocumented web APIs"** resume perfectamente cómo usar esto para hacer *scripts* de utilidad

Agenda

APIs

Explorando APIs



Consumiendo APIs

¿Quiénes consumen APIs web?

*Frontend
webapps*

*Backend
webapps*

*Mobile
apps*

*Smart devices
apps*

¿Cómo acceder a una API programáticamente?

Librerías que permitan hacer requests HTTP

Browser

XMLHttpRequest*

fetch

Node

axios

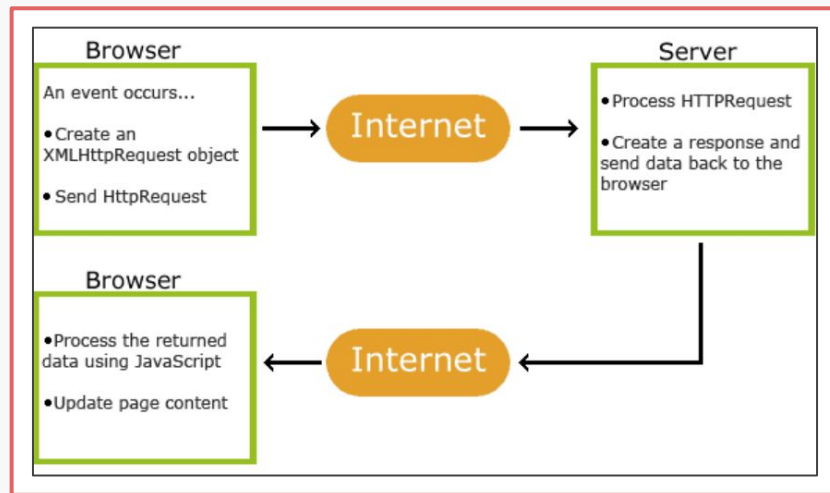
superagent

node-fetch

AJAX - *Asynchronous JavaScript and XML*

Técnica que permite realizar **requests asíncronos** en vez de síncronos

- **DOM es actualizado** cuando llega la respuesta
- Aplicación **no se bloquea** (página en blanco), se siente **más responsiva**



XMLHttpRequest

Objeto que permite utilizar Ajax. Creado por Microsoft a finales de los años 90. Hoy en día tiene una [especificación oficial](#)

```
// 1. Crear un nuevo objeto XMLHttpRequest
var xhr = new XMLHttpRequest();

// 2. Configurarlo: solicitud GET para la URL /article/.../load
xhr.open('GET', 'https://jsonplaceholder.typicode.com/posts/1', true);

// 3. Enviar la solicitud a través de la red
xhr.send();

// 4. Esto se llamará después de que se reciba la respuesta
xhr.onload = function () {
  if (xhr.status !== 200) {
    alert(`Error ${xhr.status}: ${xhr.statusText}`); // ej: error 404
  } else {
    alert(`Listo, se recibieron ${xhr.response.length} bytes`); // Respuesta
  }
};
```


Fetch

API moderna del navegador para realizar *requests* de forma asíncrona, basándose en el uso de promesas (es decir, usamos `then`, `catch`, etc.)

```
fetch('https://api.github.com/repos/IIC2513/Syllabus-S1/commits')  
  .then(response => response.json())  
  .then(commits => alert(commits[0].author.login));
```

Fetch

Si quieren ver un ejemplo de fetch en acción, recuerden que está este ejemplo en la organización en GitHub 

[IIC2513/Container.jsx](https://github.com/IIC2513/Container.jsx)

```
14
15 ... const fetchData = async () => {
16     try {
17         const response = await fetch("https://jsonplaceholder.typicode.com/todos");
18         if (response.ok) {
19             const data = await response.json();
20             // Elegimos 5 al azar
21             const filteredData = data.sort(() => Math.random() - Math.random()).slice(0, 5);
22             // Parseamos los datos
23             const parsedData = parseData(filteredData);
24
25             setList(parsedData);
26         } else {
27             throw new Error("Something went wrong while fetching data");
28         }
29     } catch (error) {
30         console.error(error);
31     }
32 }
33
```

[Axios docs](#): Librería en Node.js que permite y facilita hacer requests HTTP

AXIOS

Promise based HTTP

[Get Started](#)

client for the browser and node.js

Axios is a simple promise based HTTP client for the browser and node.js. Axios provides a simple to use library in a small package with a very extensible interface.

```
import axios from "axios";
axios.get('/users')
  .then(res => {
    console.log(res.data);
  });
```

[Get Started](#)

[View on GitHub](#)

Sponsors



Ejemplo de GET *request* a API de GitHub

Veamos un ejemplo accediendo a la lista de *commits* de un repo en GitHub ([docs](#)):

```
async function getCommits(owner, repo) {  
  // Definimos la URL del endpoint en una variable  
  const base_url = 'https://api.github.com';  
  let query_url = `${base_url}/repos/${owner}/${repo}/commits`;  
  // Se envía la request GET  
  const response = await axios  
    .get(query_url, {  
      headers: {  
        Authorization: `Bearer ${API_TOKEN}`,  
      },  
    })  
    .then(res => res.data)  
    .catch(err => console.error(err));  
  return response;  
}
```

Ejemplo de POST *request* a API de GitHub

Veamos un ejemplo para crear una *issue* en un repo en GitHub ([docs](#)):

```
async function postIssue(owner, repo) {
  // Definimos la URL del endpoint en una variable
  const base_url = 'https://api.github.com';
  let query_url = `${base_url}/repos/${owner}/${repo}/commits`;

  // Se envía la request GET
  const data = {
    title: 'Found a bug',
    body: "I''m having a problem with this.",
    assignees: ['octocat'],
    milestone: 1,
    labels: ['bug'],
  };
  const response = await axios
    .get(query_url, data, {
      headers: {
        Authorization: `Bearer ${API_TOKEN}`,
      },
    })
    .then(res => res.data)
    .catch(err => console.error(err));
  return response;
}
```

[Product](#) ▾[Pricing](#)[Enterprise](#)[Resources](#) ▾[API Network](#)[Contact Sales](#)[Sign In](#)[Sign Up for Free](#)

AI is powered by APIs. APIs are powered by Postman.

Postman is your single platform for collaborative API development. Join 35+ million devs building great APIs together, across the entire API lifecycle.

[Sign Up for Free](#)

Download the desktop app for



Send your first API request

Postman's API client enables you to create and send API [requests](#), including HTTP, GraphQL, and gRPC requests. Using Postman, you can send a request to an endpoint, retrieve data from a data source, or test an API's functionality. You don't need to enter commands in a terminal or write any code. When you create a new request and select **Send**, the API response appears right inside Postman.

Send an API request

Make sure you've [downloaded and installed the Postman desktop app](#). When you're ready, open the Postman desktop app and send your first API request.

On this page

[Send an API request](#)

[Next steps](#)



Agenda

APIs

Explorando APIs

Consumiendo APIs



¿Qué se viene pronto?

Próximas fechas importantes: clases y ayudantías

	Clases y ayudantías						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 3 (17/3 - 23/3)		JavaScript pt. 1 (Intro)		JavaScript pt. 2 (DOM y async)	Ayudantía: JavaScript		
Semana 4 (24/3 - 30/3)		React		HOY	Ayudantía: React		
Semana 5 (31/3 - 06/4)		Node.js		Desarrollando servidores	Ayudantía: Consumo de APIs		
Semana 6 (07/4 - 13/4)		Construcción de APIs		QA, CI & CD	Ayudantía: Construcción de APIs		
Semana 7 (14/4 - 20/4)		Fundamentos de front-end		Jueves Santo	Viernes Santo	Sábado Santo	Domingo de Resurrección

Próximas fechas importantes: evaluaciones

	Evaluaciones						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 3 (17/3 - 23/3)							
Semana 4 (24/3 - 30/3)	Control 1			HOY	Inicio T1 React		
Semana 5 (31/3 - 06/4)					Fin T1 (React)		
Semana 6 (07/4 - 13/4)	Control 2				Inicio T2 API		
Semana 7 (14/4 - 20/4)				Jueves Santo	Viernes Santo	Sábado Santo	Domingo de Resurrección

Clase 6

Exploración y consumo de APIs

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra
aaossa@ing.puc.cl